



Assignment

Design a Mini-Processor for BCD Arithmetic Operations

IE1044 Digital Electronics

1st year 2nd semester - 2025

ID NO: IT24103215

NAME: M S LOKU NARANGODA

Date: 18/05/2025

Introduction

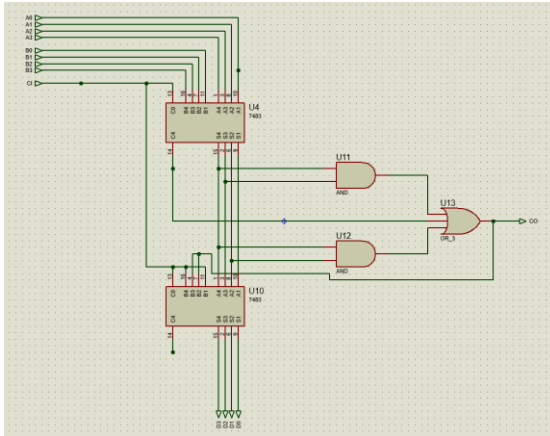
Digital Electronic is the backbone of computing today and with the arithmetic operations forming the basis of their functioning. Of all the number systems used in digital electronics, Binary Coded Decimal(BCD) gives a unique representation in which each of the decimal digits is represented by four binary bits. This scheme represents up to the decimal value 9 and higher values represent using the remaining four bits. This allows easier conversion between human-readable decimal values and machine-level binary data, making BCD particularly useful in systems like digital clocks, calculators, and financial applications where decimal precision is critical.

This assignment focuses on the design and simulation of a Mini-Processor capable of performing arithmetic operations Addition, Subtraction, and Multiplication on 2-digit BCD inputs. The objective is to deepen the understanding of BCD arithmetic, circuit-level logic design, and the integration of control logic to enable flexible operation selection based on input signals.

The designed system accepts two single-digit BCD numbers and a 2-bit control input to determine the operation to be performed. The output consists of a 2-digit BCD result along with an overflow flag to indicate if the result exceeds the 2-digit limit (i.e., greater than 99). Through this assignment, students will gain practical experience in designing individual arithmetic units, simulating their behavior using Proteus software, and integrating them into a fully functional digital processor.

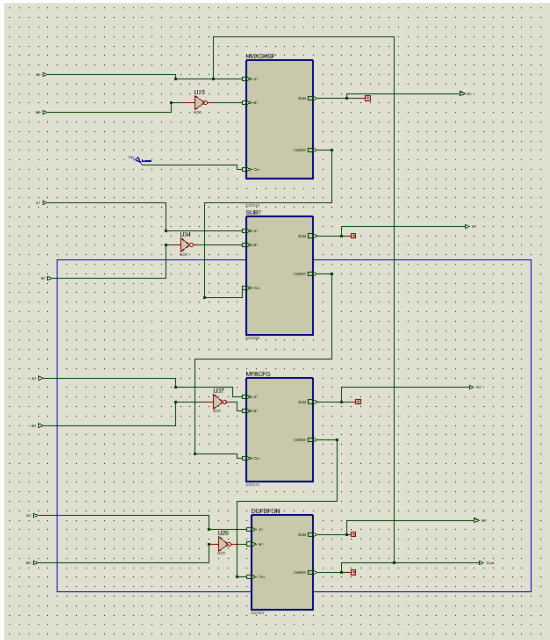
Component Design

1) BCD Adder



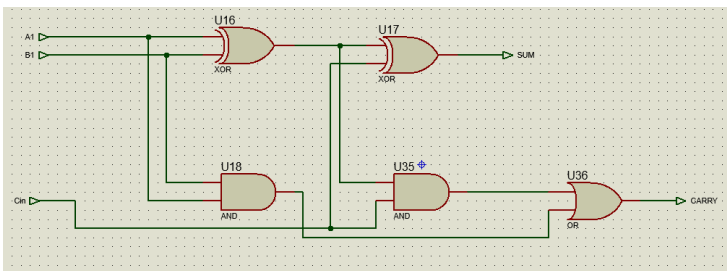
- Inside the BCD adder there are two 7483 4-bit binary full adder ICs completing the addition and by using the AND and OR gates the binary output is converted to a BCD value represented by 5 bits. This simpler method makes much easier and cleaner to represent two digit BCD value without converting the 5-bit binary value to two digit BCD converter having 4-bits in each digit.

2) Subtractor

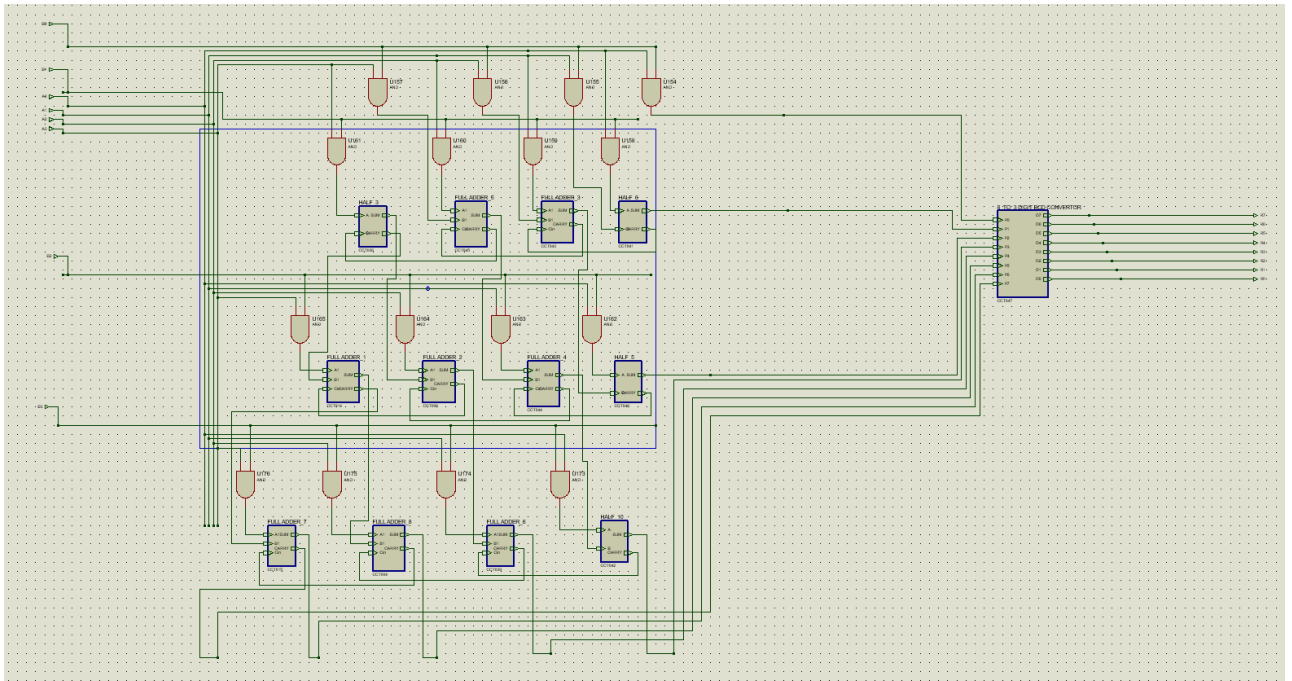


- To make the subtraction, inside the subtractor four Full Adders used and to complete the subtraction using the 2's complement, the first carry in bit was given power(5V) and the last carry out added to the sum again. The second value bits inverted and given as the second input. Inside each Full Adder, the circuit is shown below.

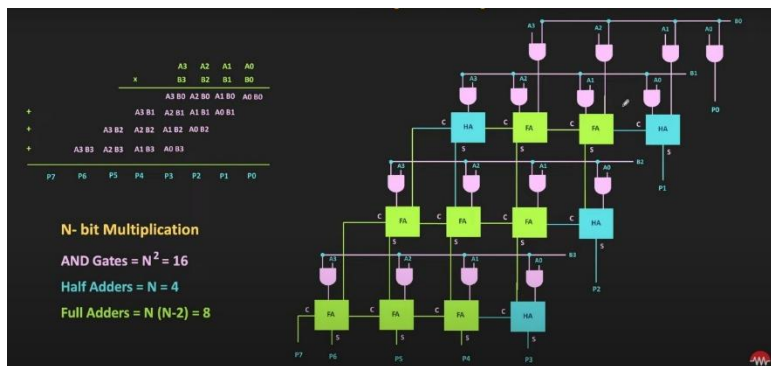
- The final binary output was not converted to BCD because by the two-digit BCD subtraction, the maximum value that can generate is also 9(9-0) and this value can be display directly without converting to BCD, making the design much simpler and having less subcircuits.



3) Multiplier

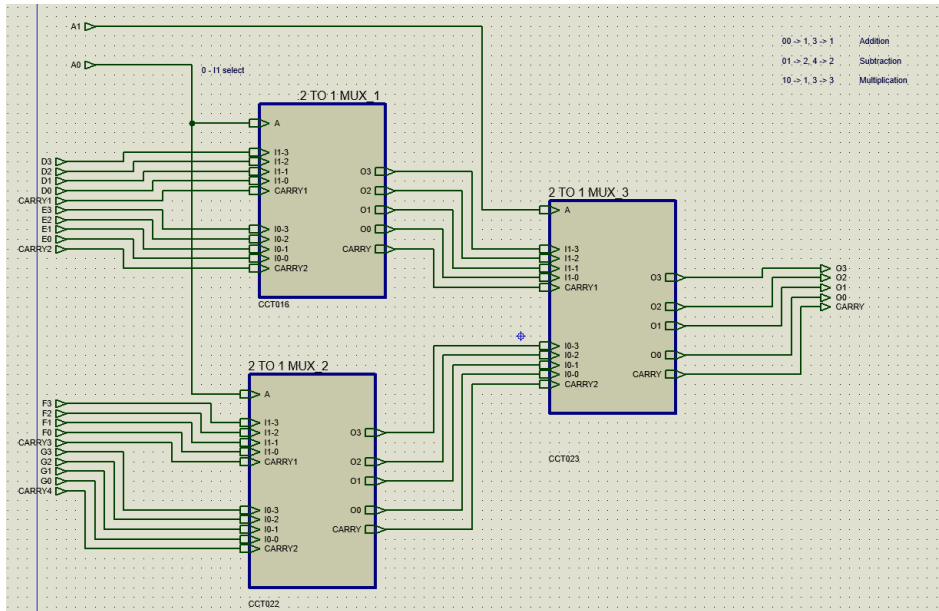


- The final objective of the project is to create a multiplier that multiplies two 4-bit decimal values. For the completion of this, the following method was used and from this, 8-bit decimal value is generated.

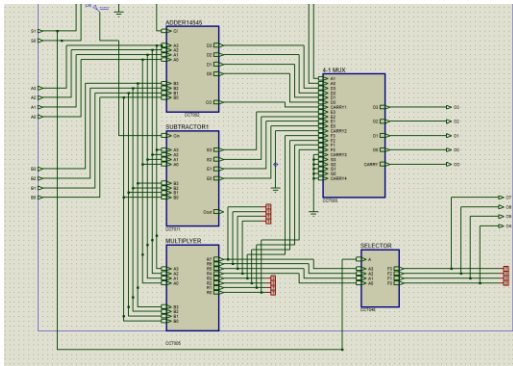


- To make 8-bit decimal values to 2-digit BCD value, 8-bit to 3 digit BCD converter was implemented, and this allows us to pass the output values directly to the two 7-segment displays.

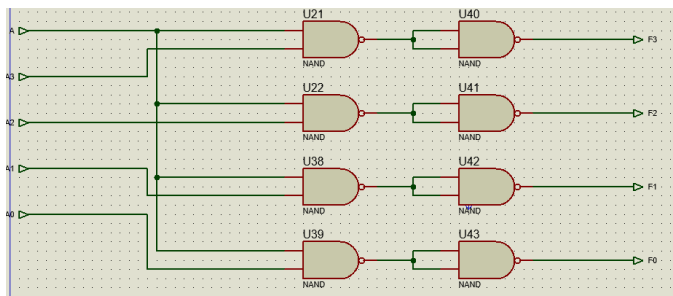
4) 4:1 Multiplexer



- To select the output from the three calculations, designed a 4:1 Multiplexer and used first three lines for the Addition, Subtraction and for the Multiplication. The remaining line was grounded as we use only the selector combinations (00, 01, 10) and three different calculations. By using this Multiplexer, the users can decide which calculation to be displayed.



- The above right hand side corner selector's purpose is to output the second BCD digit from the two digit BCD output of the multiplier only when the selector is set to multiplication without interfering the values of Addition and Subtraction.

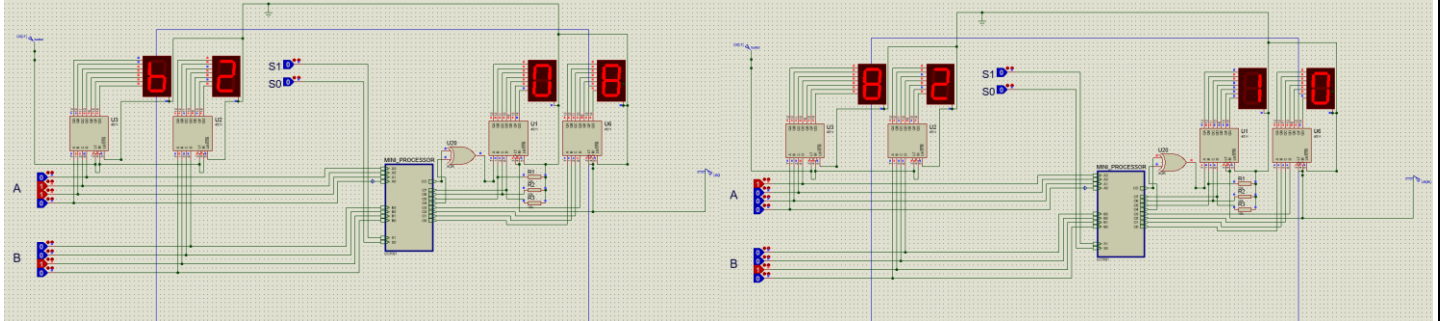


Simulations and results

1. Addition Test Cases:

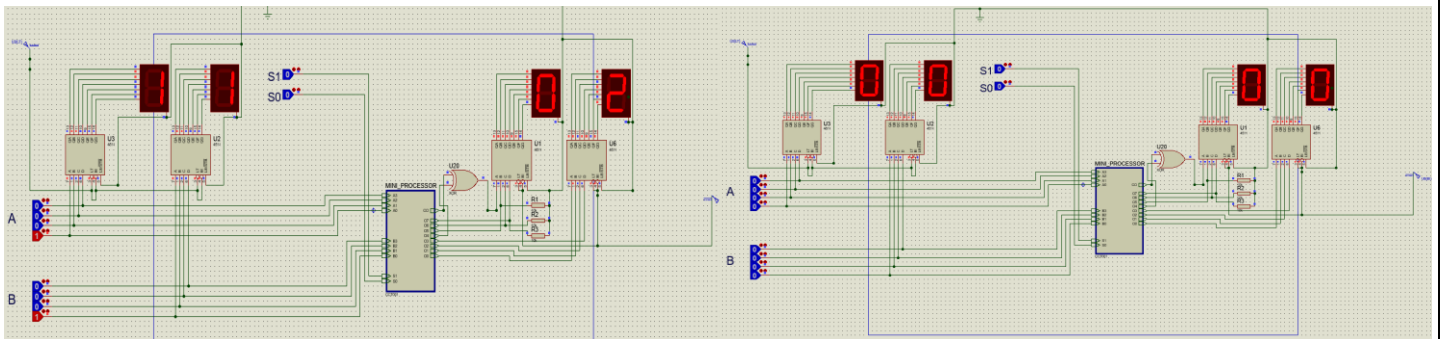
a) $6 + 2 = 8$

b) $8 + 2 = 10$

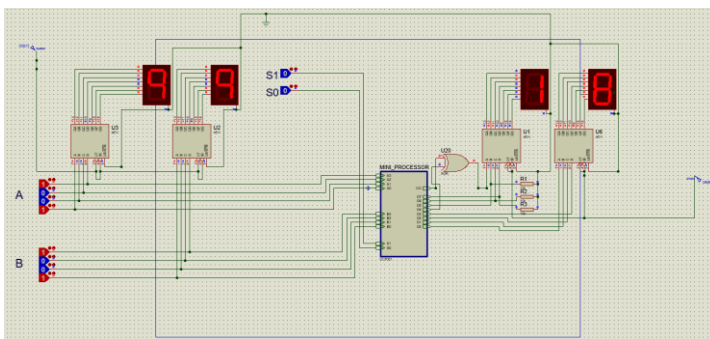


c) $1 + 1 = 2$

d) $0 + 0 = 0$

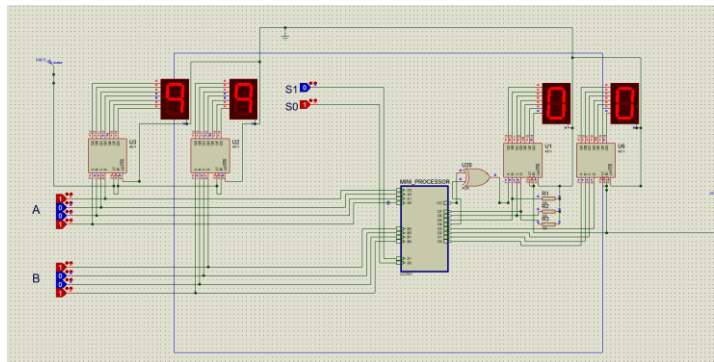


e) $9 + 9 = 18$

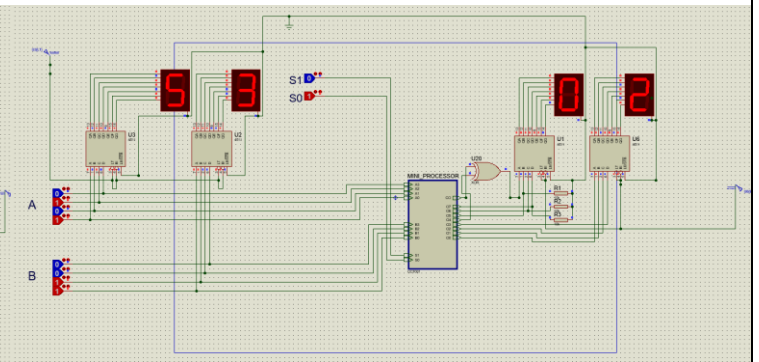


2. Subtraction Test Cases:

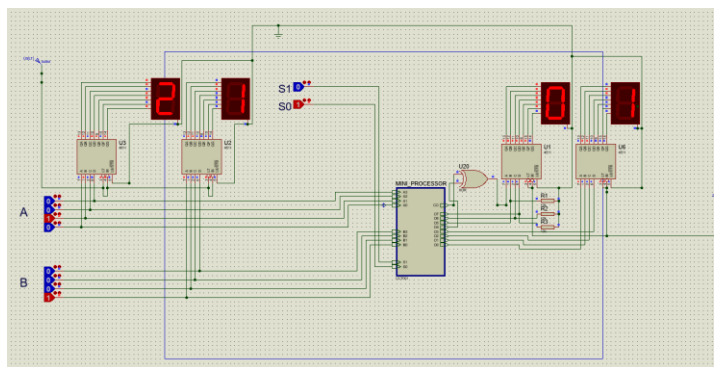
a) $9 - 9 = 0$



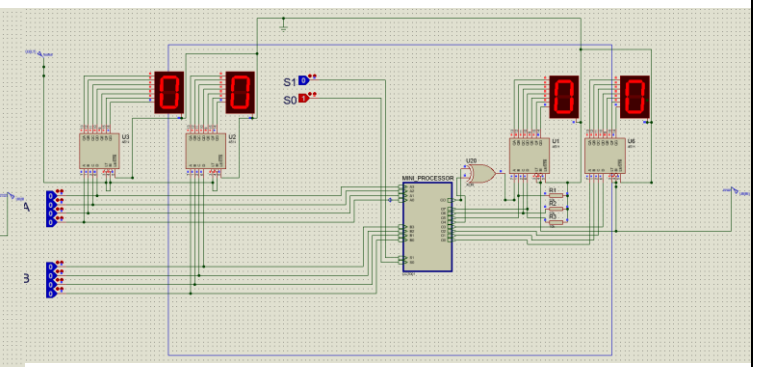
b) $5 - 3 = 2$



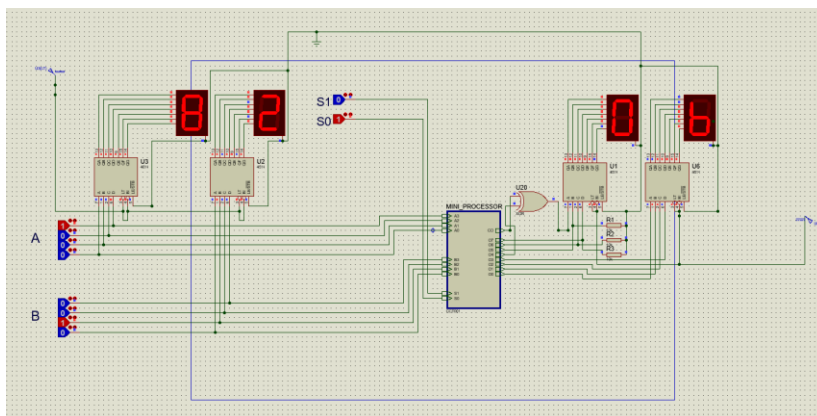
c) $2 - 1 = 1$



d) $0 - 0 = 0$



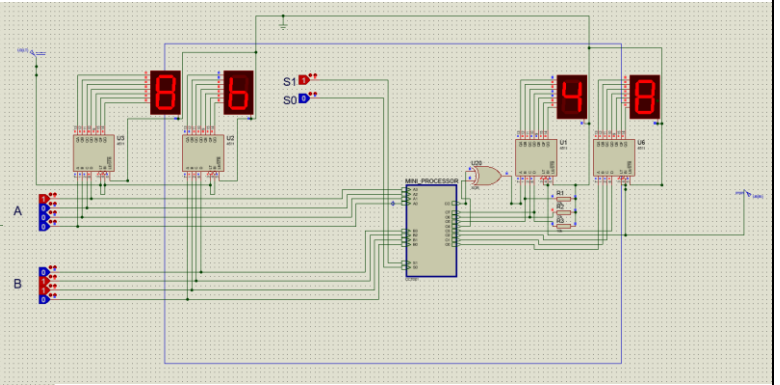
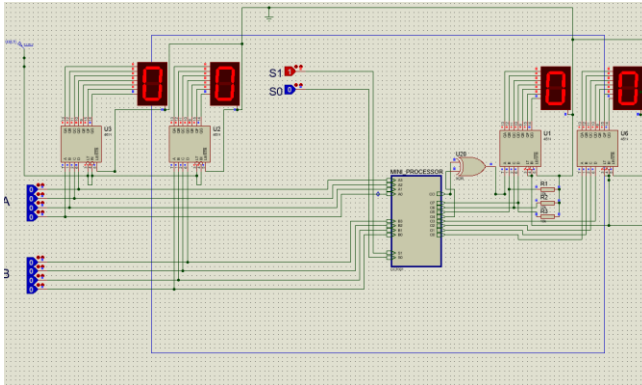
e) $8 - 2 = 6$



3. Multiplication Test Cases:

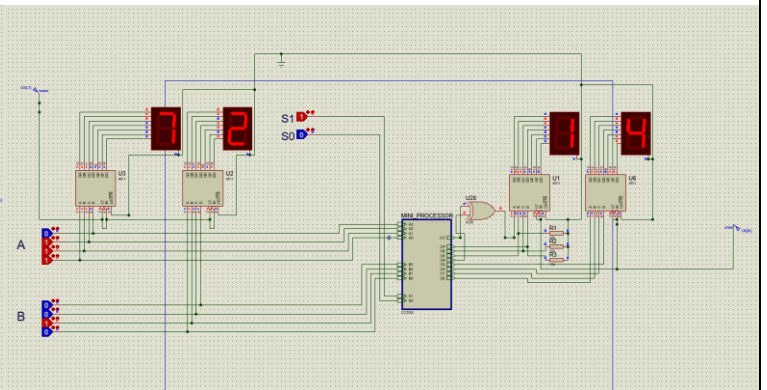
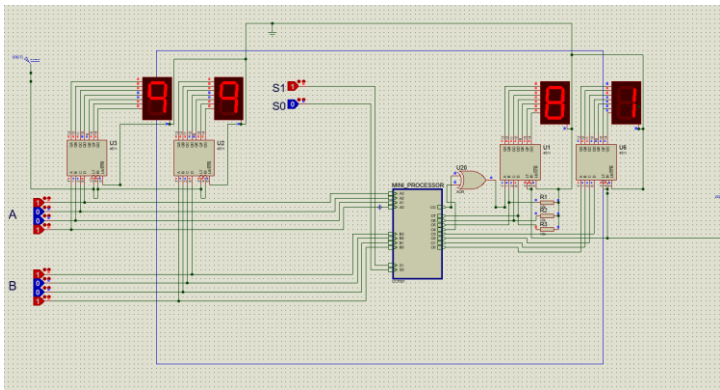
a) $0 * 0 = 0$

b) $8 * 6 = 48$

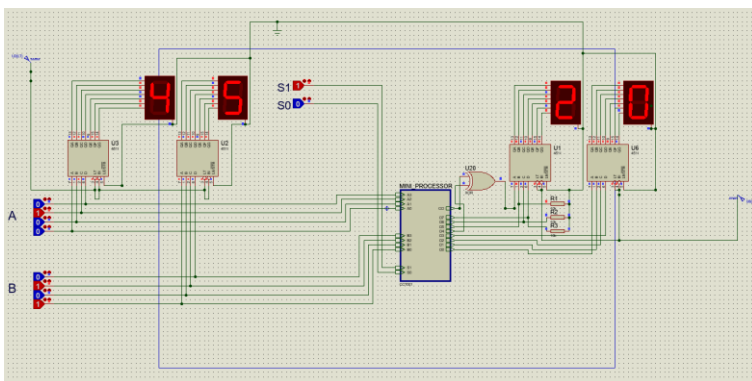


c) $9 * 9 = 81$

d) $7 * 2 = 14$



e) $4 * 5 = 20$



- Considering the above test cases in each calculation mode, we can conclude that the desired output of each is generated correctly without having any error results. This fulfils the objectives of the assignment proving that each component is working properly and the architecture of the design successfully completes the objectives.

Challenges and Solutions

1) Adapt to the new software environment:

- Getting used to a new working environment was much difficult in a short period of time. To overcome this, practiced with sample simple logic designs to have an understanding of the ICs and the tools. Referred many resources to get the knowledge of the working environment to be practiced before starting the assignment.

2) Lack of compatible ICs:

- When implementing the design of the calculation parts and other components, lack of ICs for the desired work was one of the major challenges faced. To overcome this, used ICs for the places that can be used and designed other components by hand.

3) Convert Multiplier output to 2 digit BCD:

- In the Multiplier, the output generates 8-bit binary value. To get a two digit BCD value to display each digit was a challenge because there are no clear way to implement such converter without making a custom logic gate design using the simplifications of the truth table. The truth table of such design have significantly large number of rows making it harder to design. After some research, found a method called “Double Dabble” and implemented the required converter in much easier way that saved time and the complexity of the design.

4) Error Values:

- When implementing the processor part, after completing each calculation block, checked whether each block is generating correct outputs. Each block was checked using LogicProbes and Logicstates to identify any errors in the design. This saved time by verifying the whole design at once to detect errors and detecting the error part.

Conclusion

By successfully completing the assignment objectives, got hand-on experience of the ICs and the knowledge of BCD arithmetic calculations. Also improved the understanding of logical design of a mini processor that performs the basic calculations allowing to get better basic knowledge and understanding of digital electronic designs that will be valuable when solving practical real-world problems.